

- [EKS Upgrade Guide: 1.29 → 1.31 \(Zero-Downtime\)](#)
- [Part 2: Common Errors, Troubleshooting & Rollback](#)
- [Context: Private EKS Cluster + RDS Aurora PostgreSQL](#)
 - [PRIVATE CLUSTER + AURORA SPECIFIC CONSIDERATIONS](#)
 - [Private Cluster Upgrade Challenges](#)
 - [COMMON ERRORS DURING EKS UPGRADE](#)
 - [Layer 1: Cluster \(Control Plane\) Errors](#)
 - [Layer 2: Node Group Errors](#)
 - [Layer 3: Pod Errors During/After Upgrade](#)
 - [Layer 4: Sidecar Errors](#)
 - [Layer 5: Networking Errors](#)
 - [ROLLBACK MECHANISMS](#)
 - [Rollback Decision Matrix](#)
 - [Rollback Level 1: Node Group Rollback \(Most Common\)](#)
 - [Rollback Level 2: Add-on Rollback](#)
 - [Rollback Level 3: Control Plane Rollback](#)
 - [Rollback Level 4: Aurora Connection Recovery](#)
 - [COMPLETE UPGRADE RUNBOOK \(Condensed\)](#)
 - [DOWNTIME WINDOW COMMUNICATION TEMPLATE](#)

EKS Upgrade Guide: 1.29 → 1.31 (Zero-Downtime)

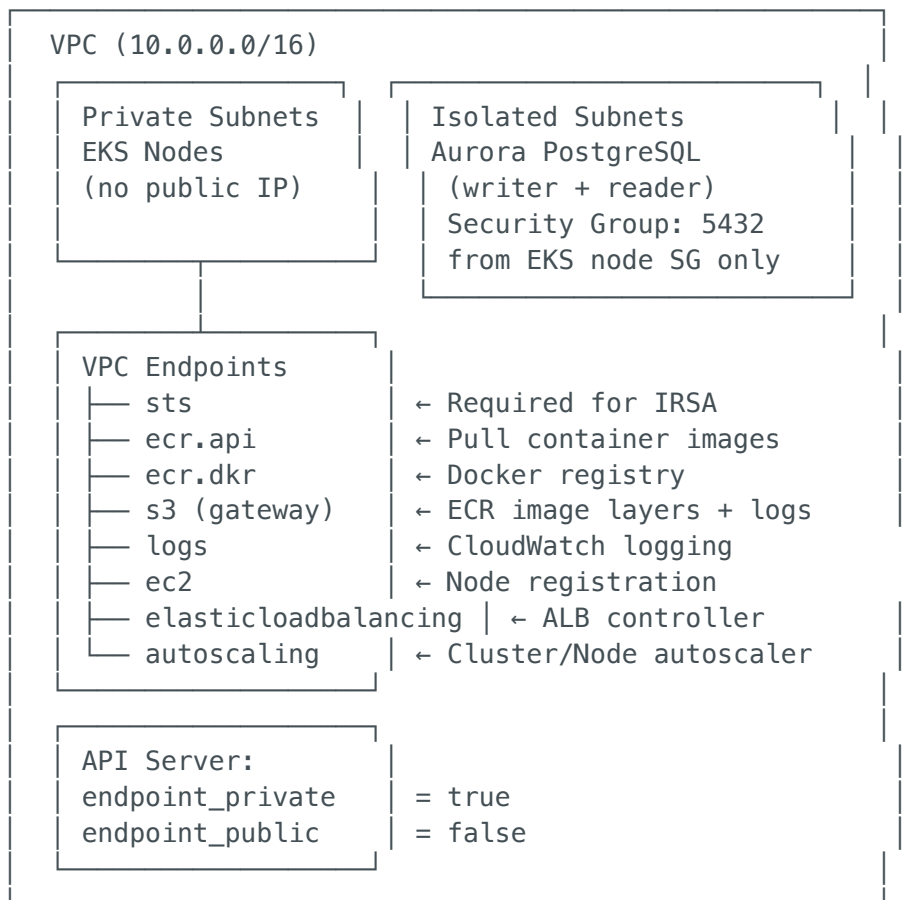
Part 2: Common Errors, Troubleshooting & Rollback

Context: Private EKS Cluster + RDS Aurora PostgreSQL

PRIVATE CLUSTER + AURORA SPECIFIC CONSIDERATIONS

Private Cluster Upgrade Challenges

Private EKS Cluster Architecture:



Private cluster upgrade requirements:

- **kubectl** access requires VPN/Direct Connect/bastion in the VPC
- CI/CD runners (GitLab) must be inside the VPC or connected via VPN
- VPC endpoints must exist for EKS to pull new AMIs and register nodes
- NAT Gateway needed if nodes pull images from public Docker Hub (prefer ECR)

Aurora-specific pre-upgrade checks:

```
# Verify Aurora connectivity from current pods
kubectl exec -it deployment/payment-service -n payments -- \
  pg_isready -h aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com -p 5432

# Check current connection count (don't upgrade if near max)
kubectl exec -it deployment/payment-service -n payments -- \
  psql $DATABASE_URL -c "SELECT count(*) FROM pg_stat_activity;"

# Check connection pool settings
kubectl get configmap payment-config -n payments -o yaml | grep -i pool
# Ensure pool settings allow for connection re-establishment during pod migration
```

COMMON ERRORS DURING EKS UPGRADE

Layer 1: Cluster (Control Plane) Errors

Error	Cause	Fix
Update is in FAILED status	SCP blocking EKS service role, or subnet IP exhaustion	Check CloudTrail for AccessDenied . Verify subnets have free IPs (minimum 5 per subnet)
Cluster is not in ACTIVE state	Previous update still running or failed	aws eks describe-cluster — wait for ACTIVE or resolve failed update
UnsupportedAvailabilityZoneException	AZ doesn't support the EKS version	Ensure cluster subnets are in supported AZs
kubectl times out after control plane upgrade	Private cluster — API server endpoint changed	Refresh kubeconfig: aws eks update-kubeconfig --name my-cluster
Unable to connect to the server post-upgrade	VPC endpoint for EKS API not configured	Create com.amazonaws.region.eks VPC endpoint
API deprecation errors in logs	Manifests using removed	Run kubent pre-upgrade, update manifests

Error	Cause	Fix
	API versions	

Debugging cluster errors:

```
# Check cluster status
aws eks describe-cluster --name my-cluster \
  --query 'cluster.{status:status,version:version,health:health}' --output json

# Check CloudTrail for upgrade failures
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=UpdateClusterVersion \
  --max-results 5

# For private clusters – verify VPC endpoint resolution
nslookup XXXXXXXX.gr7.us-east-1.eks.amazonaws.com
# Must resolve to private IP, not public
```

Layer 2: Node Group Errors

Error	Cause	Fix
Ec2SubnetInvalidConfiguration	Subnets don't have enough IPs for new nodes	Check aws ec2 describe-subnets — need IPs for new + old nodes during rolling update
NodeCreationFailure	Launch template AMI incompatible or instance type unavailable	Update launch template; check instance type availability in AZ
Node stuck in NotReady	kubelet can't reach API server (private cluster)	Verify VPC endpoints exist (sts, ec2, ecr.api, ecr.dkr, s3)
Node stuck in NotReady — CNI issue	VPC CNI can't assign pod IPs, subnet exhaustion	Check aws ec2 describe-subnets for available IPs; consider prefix delegation
Nodes are not joining the cluster	aws-auth ConfigMap	Verify: kubectl get configmap aws-auth

Error	Cause	Fix
	missing node role	<code>-n kube-system -o yaml</code>
Drain timeout	PDB too restrictive (minAvailable = replicas)	Fix PDB: <code>maxUnavailable: 1</code> and ensure replicas > 1
Taint <code>node.kubernetes.io/unreachable</code>	Node lost connectivity during upgrade	Wait for ASG to replace; check security groups

Debugging node issues:

```
# Node status with details
kubectl describe node <node-name> | grep -A 20 "Conditions:"
# Look for: Ready=False, MemoryPressure, DiskPressure, PIDPressure

# Node events (shows registration issues)
kubectl get events --field-selector involvedObject.kind=Node --sort-by='.lastTimestamp'

# kubelet logs (SSH to node via bastion in private cluster)
ssh -J bastion-host ec2-user@<node-private-ip>
sudo journalctl -u kubelet -f

# Check if node can reach API server
curl -k https://<api-server-endpoint>/healthz

# IP address availability (critical for private clusters)
for subnet in subnet-aaa subnet-bbb subnet-ccc; do
    echo "$subnet: $(aws ec2 describe-subnets --subnet-ids $subnet \
        --query 'Subnets[].AvailableIpAddressCount' --output text) IPs available"
done
```

Layer 3: Pod Errors During/After Upgrade

Error	Cause	Fix
CrashLoopBackOff after node migration	Application can't connect to Aurora	Check security group — new nodes may have different SG. Aurora SG must allow new node SG on port 5432
ImagePullBackOff	New node can't pull from ECR	Verify VPC endpoints: <code>ecr.api</code> , <code>ecr.dkr</code> , <code>s3</code> gateway. Check node IAM role has

Error	Cause	Fix
	(private cluster)	<code>ecr:GetDownloadUrlForLayer</code>
Pending — no nodes available	New nodes not ready yet, old nodes cordoned	Wait for new nodes; check ASG activity
OOMKilled after upgrade	New K8s version has slightly different memory overhead	Increase resource limits by 10-15%
Pod DNS resolution fails	CoreDNS pods restarting during add-on upgrade	Wait for CoreDNS rollout: <code>kubectl rollout status deployment/coredns -n kube-system</code>
Connection refused to Aurora	Pod moved to node in different SG or AZ	Verify Aurora SG allows the EKS node security group
too many connections on Aurora	All pods reconnect simultaneously after drain	Use connection pooling (PgBouncer sidecar or RDS Proxy)
Liveness/readiness probe failures	Probe endpoints not ready during pod startup on new node	Add <code>initialDelaySeconds: 30</code> and <code>failureThreshold: 5</code>

Debugging pod issues:

```
# Pod status and events
kubectl describe pod <pod-name> -n <namespace>

# Pod logs (current + previous container)
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous # If CrashLoopBackOff

# Test Aurora connectivity from a debug pod
kubectl run db-debug --image=postgres:15-alpine --rm -it -- \
  psql -h aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com \
  -U appuser -d appdb -c "SELECT 1;"

# Check DNS resolution (critical after CoreDNS upgrade)
kubectl run dns-debug --image=busybox:1.36 --rm -it -- \
  nslookup aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com

# Check if pod can reach Aurora endpoint
```

```
kubectl run net-debug --image=busybox:1.36 --rm -it -- \
nc -zv aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com 5432
```

Aurora connection storm prevention:

When nodes are drained, all pods restart simultaneously → connection storm to Aurora.

```
# Solution 1: Use RDS Proxy (recommended for production)
# RDS Proxy pools connections and handles connection storms
# Terraform:
resource "aws_db_proxy" "aurora_proxy" {
  name                = "aurora-proxy"
  engine_family       = "POSTGRESQL"
  vpc_subnet_ids     = var.private_subnet_ids
  vpc_security_group_ids = [aws_security_group.rds_proxy.id]
  auth {
    auth_scheme = "SECRETS"
    secret_arn  = aws_secretsmanager_secret.db_creds.arn
    iam_auth    = "REQUIRED"
  }
}

# Solution 2: PgBouncer sidecar
# Add to pod spec:
containers:
- name: pgbouncer
  image: edoburu/pgbouncer:1.21.0
  ports:
    - containerPort: 6432
  env:
    - name: DATABASE_URL
      value: "postgres://user:pass@aurora-endpoint:5432/appdb"
    - name: POOL_MODE
      value: "transaction"
    - name: MAX_CLIENT_CONN
      value: "100"
    - name: DEFAULT_POOL_SIZE
      value: "20"
```

Layer 4: Sidecar Errors

Error	Cause	Fix
Envoy/Istio sidecar CrashLoopBackOff	Service mesh version incompatible with new K8s	Upgrade Istio/App Mesh before or immediately after control plane

Error	Cause	Fix
Sidecar not injected after upgrade	Webhook certificate expired or mutating webhook broken	Check: <code>kubectl get mutatingwebhookconfigurations</code> — recreate if broken
mTLS failures between services	Certificate rotation during upgrade	Restart Istio/mesh control plane; check cert-manager
Fluent Bit sidecar OOM	New K8s version generates more log volume	Increase Fluent Bit memory limit
X-Ray daemon connectivity failure	VPC endpoint for X-Ray missing in private cluster	Create <code>com.amazonaws.region.xray</code> VPC endpoint
Init container race (pre-1.28)	Sidecar not ready before main container starts	Upgrade to native sidecars (K8s 1.29+ with <code>restartPolicy: Always</code> on init containers)

Sidecar compatibility check before upgrade:

```
# Check Istio compatibility
istioctl version
# Istio 1.20+ supports K8s 1.28–1.31
# Istio 1.22+ required for K8s 1.31

# Check all webhook configurations
kubectl get mutatingwebhookconfigurations
kubectl get validatingwebhookconfigurations
# Webhooks pointing to deleted or unhealthy pods will block pod creation

# Check cert-manager (manages sidecar certs)
kubectl get certificates --all-namespaces
kubectl get certificaterequests --all-namespaces | grep -v Approved
```

Layer 5: Networking Errors

Error	Cause	Fix
Pods can't communicate	VPC CNI version incompatible with	Upgrade VPC CNI add-on first

Error	Cause	Fix
across nodes	new K8s	
dial tcp: lookup ... no such host	CoreDNS pods crashed during upgrade	Check: <code>kubectl get pods -n kube-system -l k8s-app=kube-dns</code>
ALB Ingress 502/504 errors	ALB target group deregistration too slow	Set <code>deregistrationDelay: 30</code> on target group; add <code>preStop</code> hook
Service endpoints empty	Endpoint controller lag during API server upgrade	Wait 2-3 min; verify: <code>kubectl get endpoints <service-name></code>
Network policies not enforced	Calico/Cilium version incompatible	Upgrade CNI plugin after control plane
Pod IP exhaustion	Subnet running out of IPs during blue-green node swap	Enable VPC CNI prefix delegation: <code>ENABLE_PREFIX_DELEGATION=true</code>
Cross-AZ connectivity issues	Security group rules not updated for new nodes	Verify SG rules reference SG IDs, not IP ranges
Private cluster — ECR pull failure	Missing VPC endpoints	Need: <code>ecr.api</code> , <code>ecr.dkr</code> , <code>s3</code> (gateway), <code>sts</code> endpoints

Networking debug commands:

```
# DNS resolution test
kubectl run dns-test --image=busybox:1.36 --rm -it -- nslookup kubernetes.default
kubectl run dns-test --image=busybox:1.36 --rm -it -- nslookup payment-
service.payments.svc.cluster.local

# Pod-to-pod connectivity
kubectl exec -it deployment/app-a -n ns-a -- curl -s http://app-b.ns-
b.svc.cluster.local:8080/health

# Pod-to-Aurora connectivity
kubectl exec -it deployment/payment-service -n payments -- \
  nc -zv aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com 5432

# VPC CNI status
kubectl get pods -n kube-system -l k8s-app=aws-node
kubectl logs -n kube-system -l k8s-app=aws-node --tail=50

# Check IP allocation
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}
```

```
{.status.allocatable.pods}{"\n"}{end}'

# ALB health during upgrade
aws elbv2 describe-target-health \
  --target-group-arn $TG_ARN \
  --query 'TargetHealthDescriptions[].
{Target:Target.Id,Health:TargetHealth.State}'
```

Graceful ALB draining (prevent 502s during pod migration):

```
# Add to your Deployment spec
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
      - name: app
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 15"]
      # This gives ALB 15 seconds to deregister the target
      # before the pod starts shutting down
```

ROLLBACK MECHANISMS

Rollback Decision Matrix

Severity	Symptom	Action
LOW	Few pods restarting, self-recovering	Monitor — K8s self-heals
MEDIUM	Some services degraded, not all	Pause node drain, fix issue, continue
HIGH	Core services down (payments, DB connectivity)	Rollback node group immediately
CRITICAL	API server unresponsive, cluster unstable	Rollback control plane (AWS Support)

Rollback Level 1: Node Group Rollback (Most Common)

If blue-green approach was used:

```
# Simply delete the new node group and uncordon old nodes
# Old nodes still exist, just cordoned

# 1. Uncordon old nodes
for node in $(kubectl get nodes -l eks.amazonaws.com/nodegroup=my-node-group -o name); do
    kubectl uncordon $node
done

# 2. Drain new (problematic) nodes
for node in $(kubectl get nodes -l eks.amazonaws.com/nodegroup=my-node-group-v130 -o name); do
    kubectl drain $node --ignore-daemonsets --delete-emptydir-data --timeout=300s
done

# 3. Delete new node group
aws eks delete-nodegroup --cluster-name my-cluster --nodegroup-name my-node-group-v130

# 4. Verify pods back on old nodes
kubectl get pods --all-namespaces -o wide
```

If rolling update was used (old nodes already terminated):

```
# Create a new node group with the PREVIOUS K8s version AMI
# Note: This works because control plane supports N-1 node version

aws eks create-nodegroup \
    --cluster-name my-cluster \
    --nodegroup-name rollback-node-group \
    --kubernetes-version 1.29 \
    --node-role-arn $NODE_ROLE_ARN \
    --subnets $SUBNET_IDS \
    --instance-types t3.xlarge \
    --scaling-config minSize=3,maxSize=10,desiredSize=3

# Drain the 1.30 nodes
# Pods will schedule on 1.29 rollback nodes
```

Rollback Level 2: Add-on Rollback

```
# Rollback add-on to previous version
aws eks update-addon \
    --cluster-name my-cluster \
    --addon-name coredns \
    --addon-version v1.11.1-eksbuild.6 \
    --resolve-conflicts OVERWRITE

# If add-on is completely broken, delete and re-create
```

```
aws eks delete-addon --cluster-name my-cluster --addon-name coredns
# Re-install with known-good version
aws eks create-addon \
  --cluster-name my-cluster \
  --addon-name coredns \
  --addon-version v1.11.1-eksbuild.6
```

Rollback Level 3: Control Plane Rollback

⚠ IMPORTANT: AWS does NOT support control plane downgrade. You CANNOT go from 1.30 back to 1.29.

Mitigation strategies:

```
# Option A: Restore from Velero backup to a NEW 1.29 cluster
# 1. Create new EKS 1.29 cluster
aws eks create-cluster --name my-cluster-rollback --kubernetes-version 1.29 ...

# 2. Restore workloads from Velero backup
velero restore create --from-backup pre-upgrade-1.29-backup \
  --include-namespaces payments,inventory,notifications

# 3. Update Route53/ALB to point to new cluster
# 4. Verify Aurora connectivity from new cluster pods

# Option B: Fix forward (preferred – faster than rebuilding)
# Control plane 1.30 is running – fix the actual issue:
# – Update incompatible manifests
# – Fix add-on versions
# – Update node groups
# – Fix security group rules for Aurora connectivity
```

Rollback Level 4: Aurora Connection Recovery

```
# If pods lose Aurora connectivity after upgrade:

# 1. Verify Security Group allows new node SG
aws ec2 describe-security-groups --group-ids sg-aurora-xxxx \
  --query 'SecurityGroups[].IpPermissions[?FromPort==`5432`]'
# Ensure the EKS node security group is in the inbound rules

# 2. If SG issue – add rule immediately
aws ec2 authorize-security-group-ingress \
  --group-id sg-aurora-xxxx \
  --protocol tcp --port 5432 \
  --source-group sg-eks-nodes-new-xxxx

# 3. If connection pool exhaustion – restart pods gradually
kubectl rollout restart deployment/payment-service -n payments
```

```
# 4. If using RDS Proxy – connections recover automatically
# RDS Proxy maintains persistent connections to Aurora
# Even if all pods restart, proxy handles the storm

# 5. Emergency: Force Aurora failover to reader (if writer is overwhelmed)
aws rds failover-db-cluster --db-cluster-identifier my-aurora-cluster
# Failover takes ~30 seconds, reader promotes to writer
```

COMPLETE UPGRADE RUNBOOK

(Condensed)

EKS 1.29 → 1.31 RUNBOOK Private Cluster + Aurora DB

T-3 DAYS: PRE-CHECKS

- ☐ Run kubent – fix deprecated APIs
- ☐ Verify PDBs on all critical services
- ☐ Verify replicas ≥ 2 on all services
- ☐ Check subnet IP availability (need 2x during swap)
- ☐ Verify VPC endpoints exist (ecr, sts, s3, logs, ec2)
- ☐ Verify Aurora SG allows EKS node SG on 5432
- ☐ Check Aurora connection count (headroom for storm)
- ☐ Test Velero backup + restore in staging

T-1 DAY: STAGING UPGRADE

- ☐ Run full upgrade on staging cluster (1.29→1.30→1.31)
- ☐ Validate all applications + Aurora connectivity
- ☐ Run E2E test suite against staging
- ☐ Note any issues and prepare fixes for production

T-0: PRODUCTION UPGRADE (in maintenance window)

HOP 1: 1.29 → 1.30

- ☐ Velero backup: pre-upgrade-1.29-backup
- ☐ Control plane upgrade (25-40 min – pods unaffected)
- ☐ Upgrade add-ons: CoreDNS → kube-proxy → VPC CNI
- ☐ Create new 1.30 node group (blue-green)
- ☐ Wait for new nodes: Ready
- ☐ Cordon old nodes
- ☐ Drain old nodes one-by-one (respecting PDBs)
- ☐ After each drain: verify Aurora connectivity
- ☐ After each drain: verify app health endpoints
- ☐ All pods on new nodes → delete old node group
- ☐ VALIDATION GATE: all checks pass → proceed

HOP 2: 1.30 → 1.31

- ☐ Velero backup: pre-upgrade-1.30-backup
- ☐ Repeat all steps from HOP 1
- ☐ VALIDATION GATE: all checks pass → done

T+1: POST-UPGRADE

- ☐ Compare metrics with pre-upgrade baseline
- ☐ Verify Aurora connection pool stability
- ☐ Delete old Velero backups after 30 days
- ☐ Update documentation with new versions
- ☐ Share lessons learned with team

DOWNTIME WINDOW COMMUNICATION TEMPLATE

Subject: Planned Maintenance – EKS Cluster Upgrade

Date: [DATE] | Window: 02:00 AM – 06:00 AM IST

What: Kubernetes cluster upgrade from v1.29 to v1.31

Impact: Services remain available. Brief intermittent latency spikes possible during pod migration (< 30 seconds per service).
Database (Aurora) remains fully operational.

If issues arise:

- Minor: Resolved within maintenance window (no extension)
- Major: Rollback to previous version within 15 minutes
- Critical: Full restore from backup within 1 hour

Contact: [On-call engineer] | Slack: #infra-upgrades

Status page: <https://status.company.com>

*Prepared for: EKS Upgrade Interview Discussion & Production Runbook Context: Private
EKS Cluster with RDS Aurora PostgreSQL Candidate: Pushparaj Naik*